Writing Converters for the Function Convert Package

Barton Willis

March 22, 2026

Contents

1 Introduction and prerequisites **2**
2 Usage and features **2**
2.1 Algorithm and implementation . . . 3
2.2 Naming and the alias system . . . 3
2.3 Built-in converters . . . 4
2.4 Self-documentation feature . . . 4
2.5 Class keys and class-based dispatch . . . 5
2.6 Class–Key utilities . . . 6
2.7 Implied converters . . . 6
2.8 Chaining multiple conversions . . . 7
2.9 Implicit dispatch via breadth-first search . . . 7
2.10 User-level converters . . . 7
2.11 Errors and debugging . . . 8
3 Writing converter functions **8**
3.1 The erfc to erf converter . . . 9
3.2 Alias dispatch . . . 9
3.3 Class key converter dispatch . . . 10
3.4 Converters for subscripted functions . . . 11
3.5 An advanced converter — Maxima language . . . 12
3.6 An advanced converter — Common Lisp . . . 15
4 Regression tests **15**
5 Best practices for writing converters **16**
6 Miscellaneous **18**
6.1 Motivation . . . 18
6.2 Using `'defrule'` as an alternative to `'function_convert'` . . . 18
6.3 Design Summary . . . 19
7 Glossary **19**

1 Introduction and prerequisites

Maxima's 'function_convert' package [12] performs semantic function-to-function conversions on Maxima [5] expressions. This tutorial explains how to extend the package's mathematical capabilities by writing your own converters. To follow this tutorial, you should

- be comfortable using Maxima and the 'function_convert' package,
- understand the internal representation of Maxima expressions, and
- be able to program Maxima in Common Lisp.

This guide is intended for users who want to create new converters. It concentrates on the converter-writing process and focuses on the elements relevant to authoring converters.

To install this package, place the file 'function_convert.lisp' in a directory on Maxima's search path and load it with 'load("function_convert");'. Eventually, this package might be migrated to Maxima's /src folder — when this happens, loading the package will not be needed.

We begin with the basic usage of the 'function_convert' package and an overview of its main features.

2 Usage and features

The most basic example use of the package is a converter that replaces each subexpression of a Maxima expression of the form $\mathrm{erfc}(X)$ by $1 - \mathrm{erf}(X)$, where erfc and erf are error functions (DLMF §7.2 [7]) and X may be any expression. This converter is built into the 'function_convert' package. To use it, enter:

```
(%i1) function_convert(erfc = erf, erfc(x));
(%o1)                              1 - erf(x)
```

Of course, the input can be any expression, including nested calls to erfc; for example

```
(%i2) function_convert(erfc = erf, x*erfc(x + erfc(z)));
(%o2)                      x (erf(erf(z) - x - 1) + 1)
```

The first argument 'erfc = erf' is called a *conversion rule* (or simply a rule), the left-hand side of a rule is the *source function* and the right-hand side 'erf' is called the *target function*. The rule 'erfc = erf' does not literally replace the source function with the target function, but it instead applies the identity $\mathrm{erfc}(x) = 1 - \mathrm{erf}(x)$ to an expression, replacing the left-hand side with the right-hand side. A *converter* is the internal function that implements a rule, for this case the converter replaces $\mathrm{erfc}(X)$ by $1 - \mathrm{erf}(X)$.

The target function is best thought of a mnemonic for recalling the identity $\mathrm{erfc}(x) = 1 - \mathrm{erf}(x)$, not as a literal function. Actually, the name of the target function could be any Maxima symbol; for example, the rule could be named 'erfc = erfc_to_erf'.

This use of the infix operator '=' to define a rule is similar to its use in the Maxima function substitute, with one difference: here the operator '=' indicates a semantic conversion rather than a literal renaming. One might argue that an arrow-like operator such as '=>' would make this distinction clearer. Using '=' instead of an arrow-like symbol, however, keeps the notation consistent with substitute and preserves arrow-like

operators for other purposes.

2.1 Algorithm and implementation

The top-level function `'function_convert'` verifies that each converter has the required form, resolves any aliases, and then calls the worker function `'function-convert'`. The worker recursively traverses the expression tree and replaces function calls according to fixed rules. This is a common pattern in Maxima: a tree walk with a local transformation at each node. It is not a pattern matcher and not a general rewrite engine; that is, transformations are hard-coded rather than expressed as rewrite rules, and the shapes of expressions are not matched against declarative patterns.

Specifically, `'function-convert'` walks the expression tree in preorder (root-first, left-to-right). When an atom is encountered, it is returned immediately and no converter is applied. For non-atomic expressions, the converter itself is applied only after the arguments have been recursively converted. Thus the traversal is top-down, while the transformation is effectively bottom-up.

For example, in the expression `'a + b*c'`, the subexpression `'b*c'` is converted before the enclosing sum is processed. We can illustrate this scheme with a rule `'algebraic = boxed'` that wraps every subexpression whose operator is `'+'`, `'*'`, or `'^'` in a consecutively numbered box. This rule is not built-in to the `'function_convert'` package, but it can be written in just a few lines of code; for this code, see Section 3.3. Two examples:

```
(%i1) function_convert('algebraic = boxed, a+b*c);
(%o1) box(box(b*c,1)+a,2)

(%i2) function_convert('algebraic = boxed, a+b*(c+d));
(%o2) box(box(b*box(d+c,3),4)+a,5)
```

Two items to note: Maxima reorders commutative operands during simplification; in particular `'c+d'` prints as `'d+c'`. Also since `'algebraic'` is a Maxima option variable (default `'false'`), it is necessary to quote the source function for this converter. Notice that individual atoms are not boxed — again, this is due to the policy that when the traversal reaches an atom, no converter is called and the atom is returned immediately.

The package is implemented in Common Lisp. It has been tested using Clozure Common Lisp [2] version 1.13 and SBCL [9] version 2.4.7.

2.2 Naming and the alias system

We've seen that the name of the target function is best thought of an alias for the identity being applied. The `'function_convert'` package has an alias system that allows the source function to be a mnemonic as well. This feature is useful for naming a rule for an identity such as

$$\Gamma(z)\Gamma(1-z) = \pi/\sin(\pi z). \tag{1}$$

Here the left-hand side of this identity is a product, so the source function for this identity must be multiplication (internally `'mtimes'`), not the gamma function. But the rule `'"*" = sin'` is hard to remember and unnatural. The alias system allows the rule for this identity to have a much more natural name. For the

identity in Eq. (1), using the alias system we can name this rule `'gamma = sin'`, for example. Actually, this rule is built-in to the `'function_convert'` package:

```
(%i1) function_convert(gamma = sin, gamma(x)*gamma(1-x));
(%o1) %pi/sin(%pi*x)

(%i2) function_convert(gamma = sin, gamma(2 + x)*gamma(1-x));
(%o2) gamma(1-x)*gamma(x+2)
```

A macro takes care of the messy details of defining an alias, along with other housekeeping duties.

2.3 Built-in converters

A converter may be marked as built-in. Built-in converters cannot be removed using any user-level function. For example, the converter `'erfc = erf'` is built-in; attempting to remove it with `'delete_converter'` produces an error:

```
(%i1) delete_converter(erfc = erf);
Cannot delete built-in converter erfc = erf.
(%o1) done
```

2.4 Self-documentation feature

Every converter function should have a docstring. The docstring is available to the user via the function `'describe_converter'`; for example

```
(%i1) describe_converter(sinc = sin);
      Converter: sinc = sin
      Type: built-in
      Docstring: Convert sinc(x) into sin(x)/x.
(%o1)                                done
```

If a user-package properly includes a docstring for a converter, once the package is loaded, that documentation will be available via the function `'describe_converter'`.

Again, a macro takes care of properly storing the user documentation. But there are converters that are specializations of a more general converter. An example is the converter `'sin = exp'` that expresses the sine function in exponential form. The `'sin = exp'` converter is a specialization of the converter that expresses all trigonometric functions in exponential form. For such converters, the documentation must be manually defined. The details are in Section 2.7

2.5 Class keys and class-based dispatch

Converter functions often need to apply the same transformation to an entire family of related operators. For example, a converter that rewrites trigonometric functions into exponential functions should apply to all six trigonometric functions. Writing six separate converters is tedious and error-prone. The *class key* mechanism provides a solution.

Purpose of class keys A *class key* is a symbolic tag (such as `':trig'` or `':hyperbolic'`) that represents a category of operators. The converter system uses these tags to perform *class-based dispatch*: a single converter registered for a class key is automatically applied to every operator belonging to that class.

Class keys classify operators, not expression types. For example, the class `':algebraic'` contains the operators `'mplus'`, `'mtimes'`, and `'mexpt'`. If the head operator of an expression belongs to the class `':algebraic'`, that does not imply that the expression itself is algebraic.

The class table The mapping from class keys to operators resides in the global hashtable `'*converter-class-table*'`. This table is

Class key	Operators
:algebraic	mplus, mtimes, mexpt
:bessel	%bessel_j, %bessel_y, %bessel_i, %bessel_k, %hankel_1, %hankel_2
mexpt	
:gamma_like	%gamma, %beta, %binomial, %double_factorial, mfactorial, $pochhammer
:hyperbolic	%sinh, %cosh, %tanh, %sech, %csch, %coth
:inequation	mequal, mlessp, mleqp, mnotequal, mgreaterp, mgeqp, $notequal, $equal
:inv_hyperbolic	%asinh, %acosh, %atanh, %asech, %acsch, %acoth
:inv_trig	%asin, %acos, %atan, %asec, %acsc, %acot
:logarithmic	%log
:trig	%sin, %cos, %tan, %sec, %csc, %cot

Table 1: Class keys and their associated operators.

Each entry associates a class key with the list of operators that belong to that class. The system assumes that *an operator belongs to exactly one class*. This ensures unambiguous dispatch: when the converter system encounters an operator, it can determine its class uniquely and apply the appropriate class-level converter.

How class dispatch works When the converter system processes an expression, it identifies the head operator of the form. It then:

(a) looks up the operator in `'*converter-class-table*'`;
(b) determines the corresponding class key (e.g., `':trig'`);
(c) checks whether a converter is registered for that class;
(d) if so, applies that converter to the entire expression.

Why class keys matter Class keys provide:

- **Abstraction**: one converter per conceptual family.
- **Extensibility**: adding a new operator to a class requires no changes to existing converters.
- **Uniformity**: guarantees consistent behavior across related operators.
- **Clarity**: the class table documents the structure of the operator space.

2.6 Class–Key utilities

There are two utility functions that give convenient access to class–key information.

From class key to operators. The function `'class-table-members'` returns the list of operator symbols associated with a given class key. For example,

```
> (class-table-members :bessel)
(%BESSEL_J %BESSEL_Y %BESSEL_I %BESSEL_K %HANKEL_1 %HANKEL_2)
```

From operator to class key. The inverse query is handled by `'converter-class-of'`, which scans the class table and returns the unique class key containing the operator, or `NIL` if the operator belongs to no class; for example

```
> (converter-class-of '%bessel_j)
 :BESSEL
```

2.7 Implied converters

When a class-level converter is defined, Maxima automatically creates specialized converters for each member of the class. These are called *implied converters*. For example, `'sin = exp'` is the implied converter corresponding to the class-level converter `':trig = exp'`. Unlike the class-level converter, the implied converter `'sin = exp'` affects only the sine function:

```
(%i1) function_convert(sin = exp, sin(x) + cos(x));
(%o1) cos(x)-((%e^(%i*x)-%e^-(%i*x))*%i)/2
```

By contrast, the class converter `':trig = exp'` rewrites all trigonometric functions:

```
(%i2) function_convert(trig = exp, sin(x) + cos(x));
(%o2) (%e^(%i*x)+%e^-(%i*x))/2-((%e^(%i*x)-%e^-(%i*x))*%i)/2
```

Implied converters inherit the behavior of their class-level converter but do not inherit its documentation. User-visible documentation for an implied converter must therefore be supplied explicitly. Documentation

for non-implied converters is established automatically when the converter is created by the macro 'define-function-converter'.

All documentation strings are stored in the hashtable '*function-convert-doc*', and the source code includes several examples of how to add entries for implied converters. The hashtable key for a converter 'f = g' is the cons cell '(f . g)'.

2.8 Chaining multiple conversions

To apply two or more conversions, put the converters into a Maxima list; for example

```
(%i1) function_convert(['sinc = 'sin, 'sin = 'exp], sinc(x)^2);
(%o1) -((%e^(%i*x)-%e^-(%i*x))^2/(4*x^2))
```

Converters are applied left-to-right.

Chaining multiple conversions is handled by the function 'function_convert'; a writer of a converter doesn't need to do anything to allow chaining to work.

2.9 Implicit dispatch via breadth-first search

The utility 'find_conversion_path' uses a breadth-first search [3] of the full converter graph to find the shortest available path between a given source and target function. When several shortest paths exist, the system selects one according to its internal graph-traversal order; the choice among equal-length paths is not user-configurable.

To perform the actual conversion, the user must manually call 'function_convert' with the (possibly) multistep path returned by 'find_conversion_path'. The function 'find_conversion_path' does not perform any conversions; it only constructs the user-level representation of the path. When no such path exists, it returns an empty Maxima list. For example:

```
(%i1) path : find_conversion_path(sin, gamma);
(%o1) [sin = sinc,sinc = gamma]

(%i2) function_convert(path, sin(x));
(%o2) x/(gamma(1-x/%pi)*gamma(x/%pi+1))
```

Writers of converters only need to know that 'function_convert' uses BFS; its internal implementation details are irrelevant.

2.10 User-level converters

The function 'register_converter' provides a user-level method to define a new converter using just the Maxima language. A disadvantage to this method is that the class-key dispatch method is not available.

A converter is specified by: (a) a name for the rule, (b) an optional alias for the rule, and (c) either a Maxima function or a Maxima lambda expression implementing the conversion. An optional documentation string may also be supplied.

Here is an example of a rule that expands noncommuative powers, but not noncommuative products of sums. It is possible to give a user-defined rule an alias, but we'll define it without:

```
(%i1) register_converter("^^" = expand, lambda([a,b], expand(a^^b)),
     "Expand noncommutative powers.");
(%o1) done

(%i2) function_convert("^^" =  expand, a.(b+c) + (1+x)^^3);
(%o2) x^^3+3*x^^2+3*x+a.(c+b)+1
```

A converter defined this way is fully integrated into the built-in converter graph, and user-defined rules are discoverable by the BFS mechanism. Also, `'describe_converter'` works on user-defined rules:

```
(%i3) describe_converter("^^" = expand);
Converter: "^^""="expand
Type: "user-defined"
Docstring: "Expand noncommutative powers."
(%o3) done
```

2.11 Errors and debugging

When a non-built-in converter is redefined, Maxima will issue a warning and redefine it, but Maxima will not allow a built-in converter to be redefined. Also when a converter doesn't exist, Maxima returns the input unchanged, but doesn't issue a warning; for example

```
(%i2) function_convert(f = g, a+b);
(%o2)                                b + a
```

When writing a new converter, if a converter doesn't trigger, the problem is likely due to either a typographical error or some confusion between Maxima nouns and verbs. To debug such problems, try tracing the Common Lisp function `'lookup-converter'`. From there, you might be able to diagnose the problem.

3 Writing converter functions

A macro `'define-function-converter'` takes care of many of the housekeeping details in writing a converter, including naming the internal function that applies the rule, optionally marking the converter as built-in, and gathering the docstring for the converter to use as user-documentation. The macro also automatically handles the details needed to make the alias system work.

The function `'function_convert'` does the work of mapping the converter over the expression tree; this allows the converter code to be relatively simple. We'll begin by examining the source code of the `'erfc = erf'` converter.

3.1 The erfc to erf converter

The source code for the erfc to erf converter is

```
(define-function-converter (%erfc %erf) (op x)
    :builtin
  "Convert erfc(x) into 1 - erf(x)."
  (declare (ignore op))
  (let ((z (car x)))
    (sub 1 (ftake '%erf z))))
```

The arguments '%erf %erfc' are the source and target functions, respectively. For this converter, the argument 'op' is bound to the source function '%erfc', but this converter doesn't use this argument, so the converter code should declare to ignore it. The argument 'x' is a Common Lisp list of the argument to the function '%erfc'.

The optional keyword argument ':builtin' declares that this converter cannot be deleted using user-level functions. To mark a rule as built-in, the keyword ':builtin' must be placed before the docstring. For a non-built-in rule, omit the ':builtin' keyword. The docstring is optional, but important because it supplies the converter with documentation.

The mathematical part of the converter is

```
(let ((z (car x)))
   (sub 1 (ftake '%erf z)))
```

This is the code that implements the identity. Because 'x' is the list of arguments to '%erfc', the converter must apply '%erf' to '(car x)' rather than to 'x' itself. If you prefer, the same logic can be expressed as

```
(sub 1 (fapply '%erf x))
```

3.2 Alias dispatch

As we discussed before, both the source and the target functions of a converter can be given an alias. This can be useful when the actual source function is a poor mnemonic for the converter name.

Again, the macro 'define-function-converter' takes care of the housekeeping for defining an alias. The best way to explain how define a converter that uses alias dispatch is by an example. Here we give a converter that expands explicit powers, but not products of sums. The source function must be 'mexpt', but we'll make 'power = expand' an alias for "^" = expand. This is how it is done:

```
(define-function-converter ((mexpt $expand) ($power $expand)) (op x)
  ($expand (fapply 'mexpt x)))
```

A rule can have at most one alias.

There are two ways to invoke this converter:

```
(%i1) function_convert("^" = expand, (a+b)*x + (1+x)^2);
(%o1) x^2+(b+a)*x+2*x+1

(%i2) function_convert(power = expand_powers, (a+b)*x + (1+x)^2);
(%o2) x^2+(b+a)*x+2*x+1
```

3.3 Class key converter dispatch

Class-key converters allow a single definition to handle an entire family of related operators, for example, all trigonometric functions. Let's illustrate this method using the trigonometric class `':trig'`. Any operator listed under the `':trig'` class key in `'*converter-class-table*'` will be handled by the same converter, without the need to define six separate rules.

Specifically, we'll show how to define a rule that rewrites all trigonometric functions in terms of sine and cosine. For this converter, we finally have a reason for the `'op'` argument — this argument will be bound to one of the six trigonometric functions. The main code uses a `'case'` statement to handle each of the six cases. The code is

```
(define-function-converter (:trig $sin_cos) (op x)
  :builtin
  "Convert all six trigonometric functions to sin/cos form."
  (let ((z (first x)))
    (case op
      (%tan (div (ftake '%sin z) (ftake '%cos z)))
      (%sec (div 1 (ftake '%cos z)))
      (%csc (div 1 (ftake '%sin z)))
      (%cot (div (ftake '%cos z) (ftake '%sin z)))
      (t (ftake op z)))))
```

This converter is registered for the class key `':trig'`, not for a particular operator. When the dispatch system encounters an expression such as `'((%tan) x)'` or `'((%csc) x)'`, it determines that the operator belongs to the `':trig'` class and invokes the converter above.

The body of the converter receives two arguments: the operator `'op'` and its argument list `'x'`. The converter then rewrites each trigonometric function into its sine-cosine form.

We conclude this section with the converter `'algebraic = boxed'` introduced in Section 2.1. This example combines class-key and alias dispatch and serves primarily to illustrate the postorder application of converters; it is not intended to have mathematical significance.

```
(defmvar $box_counter 0)
(define-function-converter ((:algebraic $boxed) ($algebraic $boxed)) (op x)
```

```
(ftake 'mlabox (fapply op x) (incf $box_counter 1)))
```

This converter increments the global variable `'box_counter'` each time it is applied. In general, converters should be pure functions without side effects.

3.4 Converters for subscripted functions

Maxima represents both the polygamma and the polylogarithm function as subscripted functions. For definitions of these functions, see DLMF §5.15 and §25.12 [7]. Writing a converter for these functions, and for subscripted functions in general, requires understanding the internal structure of these expressions, and it involves using several higher-level functions that extract their components. To illustrate this, we will build a converter that applies the polygamma half-argument identity. This identity is

$$\psi_0(z) = \tfrac{1}{2}\psi_0\left(\tfrac{z}{2}\right) + \tfrac{1}{2}\psi_0\left(\tfrac{z+1}{2}\right) + \ln(2)\,. \tag{2}$$

To construct a converter for subscripted functions, we first need to understand how Maxima represents them internally. For example, the polygamma function $\psi_n(x)$ is represented internally as `'((MQAPPLY SIMP) (($PSI SIMP ARRAY) $N) $X)'`. We say that `'$psi'` is the *name* of the function; the Common Lisp list `'($n)'` is its *subscript*, and the Common Lisp list `'($x)'` is its *argument*.

All subscripted functions have `'mqapply'` as their head operator. Consequently, the target function for a converter for a subscripted function must be `'mqapply'`, and this means using an alias dispatch for these cases.

Second, we need to know how to use several high-level functions that provide access the name, subscripts, and augments of a subscripted function. For any subscripted function `'e'`, we have

(a) `'(subfunname e)'` returns the name of the subscripted function.

(b) `'(subfunsubs e)'` returns a Common Lisp list of the subscripts.

(c) `'(subfunargs e)'` returns a Common Lisp list of the arguments.

(d) `'(subfunmake fn subs args)'` constructs a subscripted function with name `'fn'`, subscripts `'subs'`, and arguments `'args'`.

With these functions we can define the converter as

```
(define-function-converter ((mqapply $psi_half) ($psi $psi_half)) (op x)
"Apply the polygamma half-argument identity; see http://dlmf.nist.gov/5.5.E8"
(let* ((e (fapply op x))
       (fn (subfunname e))  (subscripts (subfunsubs e))
       (args (subfunargs e))
       (z (div (car args) 2))
       (n (car subscripts)))
 (cond ((and (eql n 0) (eq fn '$psi))
        (add (div (add (subfunmake  fn subscripts (list z))
                       (subfunmake fn subscripts (list (add (div 1 2) z))))
```

```
                2) (ftake '%log 2)))
        (t e))))
```

In the `'let*'`, we first reconstruct the entire subscripted function, and then use the functions `'subfunname'`, `'subfunsubs'`, and `'subfunargs'` to find the function name, subscripts, and arguments. After that, we check if the lone subscript is zero and if the function name is `'$psi'`, and if so, we apply the half-argument identity; if not we return the entire subscripted function.

Because the head operator of every subscripted function is `'mqapply'`, every converter defined for a subscripted function must correctly handle every subscripted function, not just the specific one that it is intended to handle. So when testing, be sure to check that your converter handles a variety of subscripted functions correctly; for example

```
(%i1) function_convert(psi = psi_half, psi[0](x));
(%o1) (psi[0](x/2)+psi[0](x/2+1/2))/2+log(2)

(%i2) function_convert(psi = psi_half, li[2](x) + psi[1](x) + psi[0](x));
(%o2) li[2](x)+psi[1](x)+(psi[0](x/2)+psi[0](x/2+1/2))/2+log(2)
```

3.5 An advanced converter — Maxima language

In this section, we build a converter that selectively applies the simplification function `'radcan'`.

Motivation Maxima's `'radcan'` function simplifies expressions involving multivalued functions, including logarithms and radicals, but it does not strictly adhere to the principal branch of these functions. A familiar example is

```
(%i1) domain : complex$

(%i2) log(x^2);
(%o2) log(x^2)

(%i3) xxx : log(x^2)/log(x);
(%o3) log(x^2)/log(x)

(%i4) xxx = radcan(xxx);
(%o4) log(x^2)/log(x) = 2

(%i5) subst(x=-1, %);
(%o5) 0 = 2
```

The last line shows that the identity $\log(x^2) = 2\log(x)$ does not hold on its natural domain $\mathbf{C}_{\neq 0}$. In situations

where one must adhere strictly to the principal branch, it is useful to build a converter that applies 'radcan' only in controlled circumstances. There are several possible strategies; here we choose to apply 'radcan' only to *products of constant factors*.

The Maxima function We begin by constructing a Maxima function that accepts the list of factors in a product, partitions them into constant and nonconstant terms, multiplies each sublist, applies 'radcan' to the constant product, and finally returns the product of the two results.

Because this converter applies to a *product*, the primary source function is multiplication, '*'. We name the target function 'radcan_constant'. At some risk of overloading the name, we also name the Maxima function that performs the conversion 'radcan_constant'. Since the converter must accept an arbitrary number of arguments, its signature must be 'radcan_constant([e])', not 'radcan_constant(e)'.

A straightforward implementation is

```
radcan_constant([e]) := block([cnst : 1, noncnst : 1],
    for ek in e do (
        if constantp(ek) then
            cnst : cnst * ek
        else
            noncnst : noncnst * ek),
    noncnst * radcan(cnst))$
```

Now we register this function as a converter:

```
register_converter("*" = radcan_constant, radcan_constant,
    "Apply radcan to the product of constant terms.")$
```

Testing the converter To demonstrate that the converter does *not* simplify the quotient $\log(x^2)/\log(x)$, we include such a factor in a product and set the domain to 'complex':

```
(%i3) domain : complex$

(%i4) xxx : 28 + (log(x^2)/log(x)) * a*2^(1/3) * 28^(2/3);
(%o4) (2^(1/3)*28^(2/3)*a*log(x^2))/log(x) + 28

(%i5) radcan(xxx);
(%o5) 2^(8/3)*7^(2/3)*a + 28

(%i6) function_convert("*" = radcan_constant, xxx);
(%o6) (2^(5/3)*7^(2/3)*a*log(x^2))/log(x) + 28
```

In '(%o5)', 'radcan' simplifies the quotient of logarithms to 2. In contrast, the converter '"*" =

`radcan_constant'` preserves the quotient, as intended.

A subtle point: redefining a converter Suppose we redefine `'radcan_constant'` so that it applies `'radcan'` to the *entire* product:

```
(%i7) radcan_constant([e]) := radcan(xreduce("*", e))$

(%i8) function_convert("*" = radcan_constant, xxx);
(%o8) (2^(5/3)*7^(2/3)*a*log(x^2))/log(x) + 28
```

Output `'(%o8)'` shows that the updated definition has not taken effect. To update the converter, we must *re-register* it:

```
(%i9) register_converter("*" = 'radcan_constant, radcan_constant,
    "Apply radcan to a product.")$
Redefining converter mtimes mequal radcan_constant

(%i10) function_convert("*" = radcan_constant, xxx);
(%o10) 2^(8/3)*7^(2/3)*a + 28
```

When a converter is registered, the package stores a lambda form of the converter function in an internal hashtable. Redefining the Maxima function does *not* update the stored lambda. Therefore, whenever a converter function is changed, the converter must be re-registered so that the stored lambda is replaced.

Efficiency Generally, it is more efficient in Maxima to add or multiply a list of expressions en masse using `'xreduce'` or `'tree_reduce'` instead of accumulating the sum or product one term at a time as does `'radcan_constant'`. Here is a version that uses `'xreduce'`:

```
radcan_constant([e]) := block([cnst : [], noncnst : []],
    for ek in e do (
        if constantp(ek) then
            push(ek,cnst)
        else
            push(ek,noncnst)),
    radcan(xreduce(''*'',cnst) * xreduce(''*'', noncnst)))
```

Notes for the reader. When a converter is registered, the package stores a compiled lambda form of the converter function in an internal table. Redefining the Maxima function does not update this stored lambda. To ensure that a revised definition is used, the converter must be re-registered so that the stored lambda is replaced.

3.6 An advanced converter — Common Lisp

In this section, we show how to build a converter that rewrites explicit products of the form $x\ \mathrm{signum}(x)$ as $|x|$. The converter does not rewrite $\mathrm{signum}(x)$ itself, because the pattern $x\ \mathrm{signum}(x)$ is not a subexpression of $\mathrm{signum}(x)$; only explicit products matching that pattern are transformed. This is where the main complication arises: the converter must call `'xgather-args-of'` to obtain those arguments whose head operator is `'%signum'`. For each such term, the converter performs a rational substitution of the form

```
ratsubst(abs(s), s*signum(s), e),
```

where `'e'` is the product being rewritten.

The definition is:

```
(define-function-converter ((mtimes mabs) (%signum mabs)) (op x)
  :builtin
  "Convert subexpressions of the form X*signum(X) into abs(X). This converter
does not rewrite signum(X) itself, since X*signum(X) is not a subexpression
of signum(X); only explicit products matching that pattern are transformed."
  (let* ((e  (fapply op x))
         (ll (xgather-args-of e '%signum)))
    (dolist (lx ll)
      (let ((s (car lx)))
        (setq e ($ratsubst (ftake 'mabs s)
                           (mul s (ftake '%signum s))
                           e))))
    ($expand e 0 0)))
```

How the converter works

(a) The converter receives the head operator `'op'` and its argument list `'x'`. The call `'(fapply op x)'` reconstructs the full expression `'e'`.

(b) The function `'xgather-args-of'` scans `'e'` for occurrences of `'%signum'` and returns a list of matches. Each element of this list contains the argument X of a subexpression $\mathrm{signum}(X)$. The `'function_convert'` package defines the function `'xgather-args-of'`.

(c) For each such argument s, the converter constructs the product $s\ \mathrm{signum}(s)$ and replaces it with $|s|$ using `'ratsubst'`. Only explicit products are rewritten; no algebraic inference is performed.

(d) After all substitutions are attempted, the expression is resimplified with a call to `'expand'`.

This converter is intentionally conservative: it rewrites only the literal pattern $X\ \mathrm{signum}(X)$, never the function signum itself.

4 Regression tests

This section describes how to write regression tests for the `'function_convert'` package and how to run them.

Test file format A regression test file is an ordinary Maxima batch file. Each test consists of *exactly two lines*:

```
input;
expected output$
```

The first line is the Maxima expression to evaluate, and the second line is the expected result. Terminating the input line with a semicolon and the expected output with a dollar sign clarifies the intent of each line, but the line terminators can be either a semicolon or a dollar sign, if you prefer. A file can have as many tests as needed. Here is an example of one test:

```
function_convert('cos = 'exp, cos(x));
(%e^(%i*x)+%e^-(%i*x))/2$
```

Running the tests If your tests are in a file `'rtest_my_function_convert'`, to run the regression tests, enter the command:

```
batch(rtest_my_function_convert, 'test);
```

The command `'batch'` with an optional second argument of `'test'`, loads the test file, evaluates each input expression, compares the actual output with the expected output, and reports any mismatches. A successful run prints no errors.

What to test

- bogus, unexpected (boolean values, strings, or …), or invalid input,
- complex numbers,
- nested expressions and multiple occurrences the source function, and
- converters applied to expressions containing unknown functions.

Writing good tests A good regression test is minimal, documents the intended behavior of the converter system, and isolates a single behavior. If a converter is based on an identity, each such converter should have an explicit test of the identity; here is an example that tests the identity $\operatorname{erfc}(x) = 1 - \operatorname{erf}(x)$:

```
erf(x) + function_convert(erfc = erf, erfc(x));
1$
```

5 Best practices for writing converters

The guidelines below help ensure that new converters integrate smoothly into the `'function_convert'` system.

- **Base converters on identities.** A converter should be based on a mathematical identity. It should not change the meaning of an expression. If a transformation is only valid under certain assumptions, the converter should check them explicitly or decline to apply the transformation. It is discouraged, but possible, to write a converter that, for example, replaces every product by a sum.
- **Include a docstring.** Each converter should have a docstring so that `'function_convert'` is self-documenting. If a converter requires a domain restriction, document it. Clear documentation prevents misuse and helps future contributors maintain the system.
- **Use clear, descriptive names.** Converter names or aliases should reflect the mathematical transformation they perform. This helps users understand the converter registry and simplifies debugging.
- **Keep converters small and focused.** A converter should perform one clear transformation, such as rewriting $\operatorname{erfc}(x)$ as $1 - \operatorname{erf}(x)$ or converting $\tan(x)$ to $\sin(x)/\cos(x)$. Small, focused converters compose more reliably and are easier to test and debug.
- **Dispatch on functions, not patterns.** Converters operate on function calls, not syntactic patterns. Let the converter receive the arguments directly and compute the replacement semantically. This avoids brittle rules and reduces unintended matches.
- **Ensure termination.** Converters should not create expressions that trigger themselves again unless the transformation is guaranteed to converge.
- **No side effects.** Converters should be functions with no side effects.
- **Write regression tests.** Each converter should have regression tests covering typical inputs, edge cases, and expressions where the converter should *not* apply. Tests help maintain stability as the system evolves.

The example that follows illustrates why a converter should be based on an identity. If it is not, the results produced by the traversal are predictable but mathematically meaningless. Say we attempt to write a converter that returns an antiderivative; our code is

```
;; This demonstrates that converters should be based on identities.
;; When they are not, the results are predictable but useless.
(define-function-converter ((:algebraic $integrate)
                            ($algebraic $integrate)) (op x)
  (let ((e (fapply op x)))
    ($integrate e '$x)))
```

Applied to this particular expanded polynomial, the converter appears to behave correctly:

```
(%i1) function_convert('algebraic = integrate, x^2+1);
(%o1) x^3/3 + x
```

This happens because the converter maps over the arguments of the sum and encounters only atoms. By the traversal rules, it returns each atom immediately, so `'integrate'` ultimately receives the entire expression `'x^2+1'`. The call to `integrate` therefore produces a proper antiderivative.

In contrast:

```
(%i2) function_convert('algebraic = integrate, x);
(%o2) x
```

Here the return-immediately-for-an-atom rule applies at the root, so the converter is never called. The result is simply the original expression, not an antiderivative.

A more striking illustration comes from comparing an unexpanded product with its expanded form:

```
(%i3) function_convert('algebraic = integrate, (x+1)*(x+2));
(%o3) (6*x^5+45*x^4+80*x^3)/120

(%i4) function_convert('algebraic = integrate, expand((x+1)*(x+2)));
(%o4) (5*x^3)/6 + 2*x
```

Here neither result is an antiderivative of the input.

This example underscores the principle that converters should express identities. When they do not, the traversal produces consistent results, but those results need not have any mathematical significance.

6 Miscellaneous

We end with a few comments about the `'function_convert'` package that are not directly related to the process of writing converter functions.

6.1 Motivation

Many systems (Maple [4], Mathematica [8], and SymPy [10]) provide built-in expansions or rewrite mechanisms, but Maxima uses an alphabet soup of functions that perform semantic function-to-function conversions; examples include `'makefact'` and `'makegamma'`. In other cases, transformations are controlled by option variables, for example, `'expintrep'`.

These names are easy to forget and are not always easy to locate in the user documentation. The `'function_convert'` package may offer a simple, uniform, and user-extensible way to perform such conversions.

6.2 Using `'defrule'` as an alternative to `'function_convert'`

Maxima's pattern-based `'defrule'` tool is an alternative to using `'function_convert'` . A simple example that is based on the identity $\mathrm{sinc}(x) = \sin(x)/x$ is (for a definition for the sinc function, see [11])

```
(%i1) matchdeclare(aa,true)$

(%i2) defrule(sinc_rule, sinc(aa), sin(aa)/aa);
(%o2) sinc_rule:marrow(sinc(aa),sin(aa)/aa)
```

```
(%i3) apply1(sinc(sinc(x)), sinc_rule);
(%o3) (x*sin(sin(x)/x))/sin(x)
```

This method works well, especially for single-use function-to-function conversions. But there are several downsides to the 'defrule' package.

First, a 'kill(all)' removes all rules defined by 'defrule', and that makes the rule system fragile and difficult to use for anything beyond disposable transformations. Changing Maxima to allow rules that cannot be removed by 'kill(all)' is, of course, a small matter of programming [6], but it would violate a long-standing design principle: users must always be able to return the system to a clean, rule-free state. In contrast, rules that are built-in to 'function_convert' are not removed by 'kill(all)'.

Second, the 'defrule' package depends on a large assortment of auxiliary function names that can be hard to remember, and it lacks a self-documentation feature.

Third, and most importantly, it is difficult to use 'defrule' to define a rule that must match two or more terms in a sum or product. The 'function_convert' package has several converters that do exactly this. These converters often rely internally on 'ratsubst' to perform semantic substitutions — something that is difficult for a rule defined by 'defrule' to accomplish. An example is the built-in rule that applies the identity $\Gamma(z)\Gamma(1-z) = \pi/\sin(\pi z)$. The rule must scan a product and look for a pair of gamma functions whose arguments sum to one. Doing this with 'defrule' is difficult to impossible, but it can be done using 'function_convert', as we saw in Section 2.2.

We end this comparison with one possible advantage of the 'defrule' package over 'function_convert' . The 'defrule' package provides several mechanisms, including 'apply1', 'apply2', and 'applyb1', that give the user control over the order in which rules are applied, whereas the 'function_convert' package uses a fixed scheme that is not under user control.

6.3 Design Summary

The function_convert system provides a small, predictable method for rewriting expressions using alternative mathematical functions. Users may define converters explicitly, either in the Maxima language or in Common Lisp. The system does not use pattern matching; each converter applies only to the functions it is defined for, avoiding the ambiguity that can arise when patterns overlap or match more broadly than intended.

Converters may be defined for individual functions or for entire classes of related functions. A class-level converter applies uniformly to every member of the class, and the system automatically creates the corresponding implied converters for the individual functions. This allows broad transformations to be expressed concisely while keeping the behavior of each converter transparent.

All converters, explicit or implied, are stored in a central registry together with user-visible documentation strings. The registry makes it easy to inspect the available conversions, understand their behavior, and extend the system in a controlled way. The overall design emphasizes clarity, modularity, and user control rather than automatic or heuristic rewriting.

7 Glossary

atom Any expression for which the Maxima predicate 'mapatom' returns 'true'. Maxima's atoms are numbers, symbols, strings, and subscripted variables (for example, 'x[1]').

bottom–up rewriting The method in which converters are applied only after all children have been recursively converted. Although the traversal is preorder, rewriting is effectively postorder.

converter A pure function associated with a specific operator or a set of operators. It receives an expression whose children have already been converted, and returns either a rewritten expression or the original expression unchanged.

expression tree The Maxima representation of an expression, viewed as a rooted tree whose internal nodes are operators and whose leaves are atoms.

operator The function symbol at the head of a non-atomic expression. For example, in `'x+y+z'`, the operator is `'+'`.

postorder A traversal scheme in which each node is visited after all of its children have been visited.

preorder traversal A traversal scheme in which each node is visited before its children (root–first, left-to-right). `'function-convert'` uses preorder visitation but applies converters only after all children have been recursively converted.

pure function A function with no side effects and no dependence on external state. Converters must be pure: they must not mutate their input or rely on evaluation.

registry The internal hash table mapping source operators to their associated converters. Lookup is performed during traversal; if no converter is registered for an operator, the expression is returned unchanged.

source function The operator for which a converter is registered.

target function The operator produced by a converter. A converter may return an expression with the same operator or a different one.

Acknowledgements The author thanks Robert Dodier, Richard Fateman, Stavros Macrakis, and Raymond Toy for their valuable guidance and constructive discussions on the Maxima mailing list [1]. Their insights and encouragement contributed significantly to the development and refinement of this work.

References

[1] Maxima mailing list archives. https://sourceforge.net/p/maxima/mailman/, 2026. Community discussions and developer guidance.

[2] Clozure Associates. *Clozure Common Lisp*. Clozure, 2026. URL https://ccl.clozure.com/. Version 1.13 or later.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4th edition, 2022.

[4] Maplesoft. The convert function. https://www.maplesoft.com/support/help/maple/view.aspx?path=convert. Accessed 2026-02-23.

[5] Maxima Development Team. *Maxima, a Computer Algebra System*. Maxima, 2026. URL https://maxima.sourceforge.io/. Version 5.47.0 or later.

[6] Bonnie A. Nardi. *A Small Matter of Programming: Perspectives on End User Computing*. MIT Press, Cambridge, MA, 1993. ISBN 9780262140557.

[7] F. W. J. Olver, A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, and B. V. Saunders, editors. *NIST Digital Library of Mathematical Functions*. National Institute of Standards and Technology, Gaithersburg, MD, 2024. URL https://dlmf.nist.gov/.

[8] Wolfram Research. Rules. https://reference.wolfram.com/language/guide/Rules.html. Accessed 2026-02-23.

[9] SBCL Development Team. *SBCL: Steel Bank Common Lisp*. Steel Bank Common Lisp, 2026. URL https://www.sbcl.org/. Version 2.x.

[10] SymPy Development Team. Term rewriting. https://docs.sympy.org/latest/modules/rewriting.html. Accessed 2026-02-23.

[11] Eric W. Weisstein. Sinc function. From MathWorld–A Wolfram Web Resource, 2024. https://mathworld.wolfram.com/SincFunction.html.

[12] Barton Willis. A maxima package for semantic function-to-function conversions. https://github.com/barton-willis/function_convert, 2026. GitHub repository.

About the Author This manual was written by Barton Willis, Professor Emeritus of Mathematics at the University of Nebraska at Kearney, a long-time contributor to the Maxima computer algebra system, and the author of the `'function_convert'` package. He has worked in symbolic computation, mathematical software, and technical documentation for many years.

Colophon This manual was typeset using the `LuaLaTeX` engine with the `extarticle` document class. The body text is set in a serif typeface chosen for clarity in technical prose. Code fragments and Maxima sessions are rendered in a monospaced font with consistent verbatim handling. Bibliographic data were managed using `bibtex` using the `natbib` style.

All examples were executed using Maxima from the current development branch together with the `'function_convert'` package also from the current development branch.

This manual is intended as a long-term reference for users and developers who wish to extend the mathematical capabilities of the `'function_convert'` package by writing custom converters.

Version 1.0
Portions adapted from the GitHub project README [12] and from the package user documentation, used with permission.